

Практическая работа 3 по Информационным технологиям

Способы адресации

Адресация — это установление соответствия между множеством объектов ячеек памяти и множеством их адресов. Фактическое обращение к данным происходит по физическому (исполнительному) адресу. Обычно физический адрес не совпадает с адресным полем команды, но зависит от него. В общем случае происходит преобразование из адресного кода в физический код - режим (способ) адресации.

Способ адресации — это режим задания операндов внутри машинной команды.

Процессорная архитектура RISC-V использует следующие способы адресации:

- 1) регистровая,
- 2) непосредственная,
- 3) базовая,
- 4) относительная.

Первые три способа (регистровая, непосредственная и базовая) определяют способы чтения и записи операндов.

Последний способ (относительно счетчика команд) определяет способ записи счетчика команд.

При **регистровой адресации** (Register Addressing, рис.1) регистры используются для всех операндов-источников (rs1, rs2) и операндов-назначений (rd). Все инструкции типа R используют именно такой режим адресации.



Рис.1 Регистровая адресация

```
add rd, rs1, rs2      # rd = rs1 + rs2
```

После дизассемблера:

```
005303b3             add x7 x6 x5
```

При **непосредственной адресации** (Immediate Addressing, рис.2) в качестве операндов наряду с регистрами (rs1, rd) используют константы (непосредственные операнды immediate).

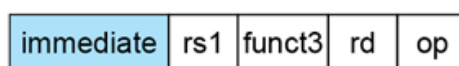


Рис.2 Непосредственная адресация

Этот режим адресации используют некоторые инструкции типа I, такие как сложение с 12-битной константой (addi) и логическая операция **andi**.

```
addi rd, rs1, 0xA     # additional immediate rd = rs1 + 0xA
```

```

andi rd, rs1, -7      # additional immediate rd = rs1 & 0xFF9

addi t2, t1, 0xA      # t2 = t1 + 10
andi t0, t1, -7       # additional immediate t0 = t1 - 7

```

Чтобы использовать константы большего размера, используется инструкция непосредственной записи в старшие разряды **lui** (load upper immediate), за которой следует инструкция непосредственного сложения **addi**.

Инструкция **lui** загружает 20-битное значение сразу в 20 старших бит и помещает нули в младшие биты:

```

lui s2, 0xABCDE       # s2 = 0xABCDE000
addi s2, s2, 0x123     # s2 = 0xABCDE123

```

После дизассемблера:

```

abcde937             lui x18 0xabcde
12390913             addi x18 x18 291

```

При использовании многоразрядных непосредственных операндов, если указанный в **addi** 12-битный непосредственный операнд отрицательный, старшая часть постоянного значения в **lui** должна быть увеличена на единицу. Помните, что знак **addi** расширяет 12-битное непосредственное значение, поэтому отрицательное непосредственное значение будет содержать все единицы в своих старших 20 битах. Поскольку в дополнительном коде все единицы означают число -1 , добавление числа, у которого все разряды установлены в 1, к старшим разрядам непосредственного операнда приводит к вычитанию 1 из этого числа. Пример иллюстрирует ситуацию, когда мы хотим в **s2** получить постоянное значение **0xFEEDA987**:

```

lui s2, 0xFEEDB       # s2 = 0xFEEDB000 (число, которое надо записать в старшие 20 разрядов (0xFEEDA), предварительно увеличено на 1)
addi s2, s2, -1657     # s2 = 0xFEEDA987 (0x987 – это 12-битное представление числа -1657) (0xFEEDB000 + 0xFFFFF987 = 0xFEEDA987)

```

При **базовой адресации** (Base Addressing, рис.3) адрес операнда в памяти вычисляется как сумма содержимого регистра (**rs1**) и 12-разрядного смещения с расширенным знаком (**immediate**).

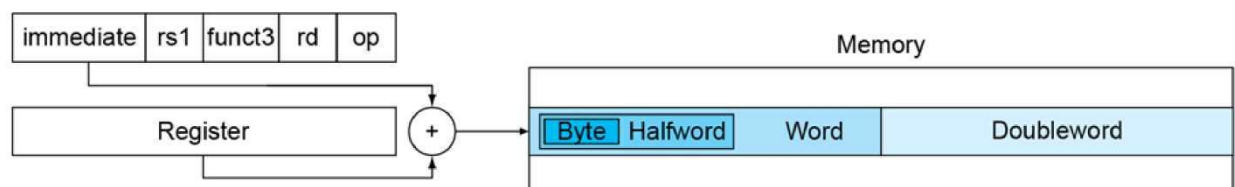


Рис.3 Базовая адресация

Базовую адресацию используют инструкции для доступа в память:

- загрузка слова (чтение памяти, **lw** – Load Word) - это инструкции типа I (Immediate);

- сохранение слова (запись в память, sw – Save Word) - инструкции типа S.

```
lw rd, 36(rs1)    # rd = M[rs1+imm][0:31]
```

Поле rs1 указывает на регистр, содержащий базовый адрес, а поле rd указывает на регистр-назначение. Поле imm, хранящее непосредственный операнд, содержит 12-битное смещение, равное 36. В результате регистр rd содержит значение из ячейки памяти rs1+36:

```
sw rs2, 8(rs1)    # M[rs1+imm][0:31] = rs2[0:31]
после дизассемблера:    0240a183    lw x3 36 x1
```

Инструкция сохранения слова sw демонстрирует запись значения из регистра rs2 в слово памяти, расположенное по адресу rs1+8.

```
После дизассемблера:    00c5a423    sw x12 8 x11
```

При **относительной адресации** (PC-relative Addressing, рис.4):

- инструкции условного перехода, или ветвления, используют адресацию относительно счетчика команд (PC) для определения нового значения PC в том случае, если нужно осуществить переход;

- смещение со знаком прибавляется к PC для определения нового значения PC, поэтому тот адрес, куда будет осуществлён переход, называют адресом относительно PC;

- инструкции условного перехода (beq, bne, blt, bge, bltu, bgeu) типа B и jal (безусловный переход и связывание) типа J используют для смещения 13- и 21-битные константы со знаком соответственно.



Рис.4 Относительная адресация

При этом самые старшие значимые биты смещения располагаются в 12- и 20-битных полях инструкций типа B и J, а наименьший значащий бит смещения всегда равен 0, поэтому он отсутствует в инструкции. Например:

```

1) beq rs1, rs2, imm    # if(rs1 == rs2) PC += imm
   после дизассемблера:  00208a63    beq x1 x2 20

2) jal rd, imm          # rd = PC+ 4; PC += imm
   после дизассемблера:  014003ef    jal x7 20

```

Инструкция jal может быть использована как для вызова функций, так и для простого безусловного перехода. Соглашение, используемое в RISC-V таково, что адрес возврата должен быть сохранён в соответствующем регистре ra (x1).

Инструкция `jal` не имеет достаточного места для кодирования полного 32-битного адреса. Например:

```
3) jalr rd, rs1,imm    # rd = PC + 4, PC = rs1 + imm
   после дизассемблера: 01408167 jalr x2 x1 20
```

Разница состоит в том, что переход `jalr` не происходит относительно `PC`. Вместо этого он происходит относительно `rs1`. Для этого можно использовать `addi` для установки значения регистра `rs1`, но на практике поступают иначе. Чтобы адрес был относителен к программному счётчику (`PC`), применяется специальная инструкция `auipc`. Инструкция `auipc` (Add Upper Immediate to Program Counter) типа U (сложить старшие разряды константы смещения с `PC`) также использует адресацию относительно счетчика команд.

```
4) auipc rd, imm       # rd = PC + (imm << 12)
   после дизассемблера: 00014117 auipc x2 0x14
```

Инструкция `auipc` типа U (сложить старшие разряды константы смещения с `PC`) также использует адресацию относительно счетчика команд. Эта инструкция является очень важной при работе с подпрограммами. Об этом подробнее будет рассказано в позже в другой практической работе.

Чтобы в ассемблере использовать именованные константы, применяется директива `.equ`. Пример:

```
.data                # сегмент инициализации данных
.equ n1 101          # n1 = 101
.equ n2 - 634        # n2 = -634
```

Задание.

Напишите четыре программы на ассемблере, использующие различные режимы адресации. Используйте последние 3 цифры вашего номера студенческого билета как число x . Например, для номера 431137: $x = 137$. Числа n зависят от x : $n_1 = x$, $n_2 = x - 635$. Константы n задайте с помощью директивы `.equ`

1. Регистровая адресация. Сложить 2 константы n_1 и n_2 . Константы разместить в регистрах `s2` и `s3`, а результат сложения - в регистре `s4`.

2. Непосредственная адресация. Переслать константу n_1 в регистр `s0` и выполнить логическую операцию И с константой -8. Результат операции сохранить в регистре `s1`.

3. Базовая адресация. Сложить n целых чисел $sum = a[0] + a[1] + a[2] + \dots + a[n-1]$, результат записать в память. Первое число $a[0]=n_2$, последующие числа $a[i] = n_2//4 + 2i$.

Регистровый файл

s2	Address of a[i]
s3	N
s4	Sum
s5	100

Память

0	a[0]
4	a[1]
8	a[2]
....	
	a[n-1]
100	0
104	N
108	sum

4. Адресация относительно счетчика команд. Найти $c = \max(n_1, n_2)$

Регистровый файл до выполнения программы

a0	n_1
a1	n_2

Регистровый файл после выполнения программы

a0	c
----	---

Вопросы для контроля

1. Что такое регистровый файл?
2. В чем отличие регистров и основной памяти?
3. Объясните методы адресации, используемые в RISC-V.